

# 20 Questions SQL

## Test de niveau /20

Raisonnement : jointures, agrégats, NULL, fenêtrage

Durée conseillée : 20 min · 3 niveaux · corrigé avec le piège de chaque question

**Objectif** : mesurer ta capacité à raisonner en SQL, pas à réciter de la syntaxe. Chaque question teste un réflexe qui sépare quelqu'un qui connaît SQL de quelqu'un qui sait résoudre un problème quand la requête renvoie un résultat bizarre. Une seule bonne réponse par question.

**Barème** : 1 point par bonne réponse, note sur 20. **12/20** = niveau junior solide. **17/20** = tu tiens un test technique sans transpirer. Note ton score, le corrigé t'explique le piège de chaque erreur.

Candidat \_\_\_\_\_ Date \_\_\_\_\_ Note \_\_\_\_\_ /20

### ■ ■ ■ Niveau 1 : Junior

**Question 1** : Tu veux ne garder que les commandes dont le montant dépasse 100. Quelle clause utilises-tu pour filtrer ces lignes *avant* toute agrégation ?

- a) `HAVING montant > 100`
- b) `WHERE montant > 100`
- c) `GROUP BY montant > 100`
- d) `FILTER montant > 100`

**Question 2** : Une table `clients` a 50 lignes, une table `commandes` a 80 lignes. Un `INNER JOIN` sur `client_id` renvoie au maximum combien de lignes, sachant que chaque commande pointe vers un seul client ?

- a) Exactement 50 lignes.
- b) Exactement 130 lignes.
- c) Au plus 80 lignes.
- d) Toujours 4000 lignes (produit cartésien).

**Question 3 :** Tu veux la liste des villes distinctes présentes dans la table `clients`. Quelle requête est correcte ?

- a) `SELECT DISTINCT ville FROM clients`
- b) `SELECT UNIQUE(ville) FROM clients`
- c) `SELECT ville FROM clients GROUP`
- d) `SELECT DISTINCT(*) ville FROM clients`

**Question 4 :** Dans quel ordre logique SQL traite-t-il ces clauses ?

- a) `SELECT`, puis `FROM`, puis `WHERE`, puis `GROUP BY`.
- b) `FROM`, puis `WHERE`, puis `GROUP BY`, puis `SELECT`.
- c) `WHERE`, puis `FROM`, puis `SELECT`, puis `ORDER BY`.
- d) `SELECT`, puis `WHERE`, puis `FROM`, puis `GROUP BY`.

**Question 5 :** Tu écris `SELECT * FROM ventes ORDER BY montant DESC LIMIT 5`. Que renvoie cette requête ?

- a) Les 5 ventes au montant le plus faible.
- b) Les 5 ventes au montant le plus élevé.
- c) Les 5 premières lignes insérées dans la table.
- d) Une erreur, `ORDER BY` et `LIMIT` sont incompatibles.

**Question 6 :** Quelle fonction d'agrégation compte le nombre total de lignes d'un groupe, y compris celles où la colonne ciblée vaut `NULL` ?

- a) `COUNT(colonne)`
- b) `COUNT(*)`
- c) `COUNT(DISTINCT colonne)`
- d) `SUM(colonne)`

**Question 7 :** Tu veux concaténer le résultat de deux `SELECT` en supprimant les doublons. Quel opérateur utilises-tu ?

- a) `UNION ALL`
- b) `UNION`
- c) `JOIN`
- d) `INTERSECT`

## ■■■ Niveau 2 : Intermédiaire

**Question 8 :** Tu fais `SELECT pays, SUM(ca) FROM ventes GROUP BY pays HAVING SUM(ca) > 1000`. Pourquoi le filtre est-il dans `HAVING` et non dans `WHERE` ?

- a) Parce que `WHERE` est plus lent que `HAVING`.
- b) Parce que `WHERE` ne peut pas filtrer sur le résultat d'une agrégation, qui n'existe qu'après le `GROUP BY`.
- c) Parce que `HAVING` s'applique avant le `GROUP BY`.
- d) Parce que `WHERE` ne fonctionne pas avec `SUM`, alors que `HAVING` ne fonctionne qu'avec `SUM`.

**Question 9 :** La table `commandes` a une colonne `remise` qui contient parfois NULL. Que renvoie `AVG(remise)` sur 3 lignes valant 10, 20 et NULL ?

- a) 10 (le NULL compte comme 0 dans la moyenne).
- b) 15 (la moyenne ignore le NULL au numérateur et au dénominateur).
- c) NULL (une seule valeur NULL rend toute la moyenne NULL).
- d) Une erreur d'exécution.

**Question 10 :** Tu veux toutes les catégories, même celles sans aucun produit associé. Quelle jointure entre `categories` (à gauche) et `produits` ?

- a) `INNER JOIN`
- b) `LEFT JOIN`
- c) `CROSS JOIN`
- d) `RIGHT JOIN` avec `categories` à gauche.

**Question 11 :** Une sous-requête corrélée comme `SELECT nom FROM emp e WHERE salaire > (SELECT AVG(salaire) FROM emp WHERE dept = e.dept)` fait quoi ?

- a) Renvoie les employés gagnant plus que la moyenne globale de l'entreprise.
- b) Renvoie les employés gagnant plus que la moyenne de *leur propre* département.
- c) Renvoie une erreur, on ne peut pas référencer `e` dans la sous-requête.
- d) Renvoie la moyenne de chaque département.

**Question 12 :** Tu veux compter combien de *valeurs différentes* de `statut` existent dans la table. Quelle expression ?

- a) `COUNT(statut)`
- b) `COUNT(*)`
- c) `COUNT(DISTINCT statut)`
- d) `DISTINCT COUNT(statut)`

**Question 13 :** Une CTE (`WITH ventes_mensuelles AS (...)`) sert principalement à quoi ?

- a) Créer une table physique stockée sur le disque.
- b) Nommer un résultat intermédiaire pour structurer et réutiliser la logique dans la requête.
- c) Accélérer systématiquement la requête en créant un index.
- d) Remplacer le `GROUP BY`.

**Question 14 :** Tu écris `ROW_NUMBER() OVER (PARTITION BY dept ORDER BY salaire DESC)`. À quoi sert le `PARTITION BY` ?

- a) Il filtre les lignes comme un `WHERE`.
- b) Il redémarre la numérotation à 1 pour chaque département.
- c) Il trie le résultat final de la requête.
- d) Il supprime les doublons de département.

## ■■■ Niveau 3 : Piège

**Question 15 :** Tu fais `SELECT * FROM clients LEFT JOIN commandes ON clients.id = commandes.client_id WHERE commandes.montant > 50`. Quel est le piège?

- a) Aucun, la requête garde bien tous les clients.
- b) Le `WHERE` sur la table de droite élimine les clients sans commande, le `LEFT JOIN` se comporte comme un `INNER JOIN`.
- c) `LEFT JOIN` et `WHERE` sont interdits ensemble.
- d) La requête renvoie un produit cartésien.

**Question 16 :** La sous-requête `SELECT client_id FROM blacklist` peut renvoyer des `NULL`. Que renvoie `SELECT * FROM clients WHERE id NOT IN (SELECT client_id FROM blacklist)`?

- a) Tous les clients non présents dans la blacklist.
- b) Aucune ligne, car un `NULL` dans la liste rend chaque comparaison `NOT IN` indéterminée.
- c) Une erreur de syntaxe.
- d) Uniquement les clients dont l'id est `NULL`.

**Question 17 :** Tu veux un total cumulé jour par jour avec `SUM(v) OVER (ORDER BY jour)`. Deux ventes tombent le même jour. Que se passe-t-il par défaut?

- a) Chaque ligne reçoit un cumul distinct dans l'ordre d'insertion.
- b) Les deux lignes du même jour reçoivent le *même* cumul, car le cadre par défaut est `RANGE` et regroupe les valeurs ex aequo.
- c) Le cumul est calculé indépendamment par jour, sans accumulation.
- d) La requête échoue car il manque `PARTITION BY`.

**Question 18 :** Sur des montants 100, 100, 90 triés en décroissant, que renvoient respectivement `RANK()`, `DENSE_RANK()` et `ROW_NUMBER()` ?

- a) 1,1,3 · 1,1,2 · 1,2,3
- b) 1,2,3 pour les trois fonctions.
- c) 1,1,2 · 1,1,3 · 1,2,3
- d) 1,1,1 · 1,1,1 · 1,2,3

**Question 19 :** Tu compares `WHERE statut = NULL` et `WHERE statut IS NULL`. Quelle affirmation est exacte?

- a) Les deux renvoient les lignes où `statut` est nul.
- b) `= NULL` ne renvoie jamais de ligne, car une comparaison avec `NULL` donne « inconnu »; il faut `IS NULL`.
- c) `IS NULL` est obsolète, on doit écrire `= NULL`.
- d) `= NULL` renvoie toutes les lignes de la table.

**Question 20 :** Tu veux le 2e salaire le plus élevé par département. Pourquoi `MAX(salaire)` avec `GROUP BY dept` ne suffit-il pas, et que faut-il utiliser?

- a) `MAX` donne déjà le 2e; aucune autre fonction n'est nécessaire.
- b) `MAX` ne donne que le 1er; il faut une fonction de fenêtrage comme `DENSE_RANK()` (ou `ROW_NUMBER()`) partitionnée par `dept`, puis filtrer le rang 2.
- c) Il faut `SUM(salaire) / 2`.
- d) Il faut deux `GROUP BY` imbriqués sur la même colonne.

## ■ ■ ■ Corrigé et explications

**Rappel du barème :** 1 point par bonne réponse, sur 20. **12/20** = niveau junior solide. **17/20** = tu tiens un test technique sans transpirer.

- **1 : b)** WHERE filtre les lignes avant l'agrégation. HAVING filtre les groupes après. Piège : filtrer un agrégat dans WHERE est une erreur.
- **2 : c)** Chaque commande pointe vers un client, donc l'INNER JOIN renvoie au plus une ligne par commande : au plus 80.
- **3 : a)** SELECT DISTINCT ville retire les doublons. UNIQUE n'est pas une fonction SQL standard.
- **4 : b)** L'ordre logique est FROM puis WHERE puis GROUP BY puis HAVING puis SELECT puis ORDER BY. C'est pour ça que WHERE ne voit pas les alias du SELECT.
- **5 : b)** ORDER BY ... DESC puis LIMIT 5 renvoie les 5 montants les plus élevés.
- **6 : b)** COUNT(\*) compte toutes les lignes. COUNT(colonne) ignore les NULL de cette colonne.
- **7 : b)** UNION déduplique, UNION ALL garde les doublons (et est plus rapide).
- **8 : b)** WHERE s'exécute avant le GROUP BY : l'agrégat n'existe pas encore. Seul HAVING peut filtrer sur SUM(ca).
- **9 : b)** Les fonctions d'agrégation comme AVG ignorent les NULL : moyenne de 10 et 20 = 15, pas 10. Le NULL n'est compté ni au numérateur ni au dénominateur.
- **10 : b)** Le LEFT JOIN conserve toutes les lignes de la table de gauche, ici toutes les catégories, même sans produit.
- **11 : b)** La sous-requête corrélée se réévalue par ligne : elle compare chaque employé à la moyenne de son propre département via e.dept.
- **12 : c)** COUNT(DISTINCT statut) compte les valeurs distinctes. COUNT(statut) compterait toutes les valeurs non nulles, doublons inclus.
- **13 : b)** Une CTE nomme un résultat intermédiaire pour clarifier et réutiliser la logique. Ce n'est pas une table physique ni un index.
- **14 : b)** PARTITION BY découpe les lignes en groupes : la numérotation redémarre à 1 dans chaque département. Il ne filtre pas, contrairement à WHERE.
- **15 : b)** Piège classique. Un filtre sur la table de droite dans le WHERE élimine les lignes NULL issues du LEFT JOIN : il redevient un INNER JOIN. Mettre la condition dans le ON préserve les clients sans commande.
- **16 : b)** NOT IN avec un NULL dans la sous-requête renvoie « inconnu » pour chaque ligne, donc aucune ligne ne ressort. Préférer NOT EXISTS ou exclure les NULL.
- **17 : b)** Sans cadre explicite, OVER (ORDER BY jour) utilise RANGE : les lignes ex aequo sur jour partagent le même cumul. Pour un vrai pas à pas, ajouter ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW.
- **18 : a)** RANK laisse un trou après l'ex aequo (1,1,3), DENSE\_RANK non (1,1,2), ROW\_NUMBER ignore les ex aequo (1,2,3).
- **19 : b)** statut = NULL est toujours « inconnu », donc ne renvoie jamais de ligne. La seule forme correcte est IS NULL.
- **20 : b)** MAX ne donne que le 1er. Pour le 2e par groupe, on numérote avec DENSE\_RANK() (ou ROW\_NUMBER()) PARTITION BY dept ORDER BY salaire DESC, puis on garde le rang 2.